Day 8 — Live coding: replacing the lies with Python

Day 7 ended with two confessed lies: a balance invented by a `set_slots` line, and a "your appointment is booked" that booked nothing. Today we pay that debt. Python enters the project through **custom actions** — and we use three of them to meet the three faces of the SDK: what an action *returns* (events), what it *reads* (the tracker), and what it *sends* (the dispatcher). Along the way we run a mock core-banking API, wire the action code in two different ways, and stage both a designed failure and an unhandled one.

The day's through-line: **logic in flows, work in actions**. An action fetches facts and reports whether the fetch worked; the flow decides what the assistant says next.

The end state ships in this folder: `lumera-assistant/` (the project as this tutorial leaves it), `lumera-fastapi-server/` (the mock bank), and a `Makefile` that runs the processes.

> **REQUIREMENTS** - the Banca Lumera project exactly as Day 7 left it (or copy this folder's `lumera-assistant/` and delete the three `actions/*.py` files to follow along) - `RASA_LICENSE` and `OPENAI_API_KEY` exported in your shell - two more variables, used by the integration (the `Makefile` sets them for processes it starts; export them yourself in any terminal where you run `rasa` directly):
>
> ```
> export CORE_BANKING_URL=http://localhost:8000
> export CORE_BANKING_TOKEN=test_token
> ```
>
> - the assistant's Python dependencies (`rasa-pro` and `requests`) — no manual step: `make setup` (§2) builds the venv for you, whichever starting point you took above

1. Actions were running all along

Before writing one, notice that yesterday's "no Python" project was already full of actions. Every turn ends in `action_listen` (wait for input). Every session opens with `action_session_start`. Every response we triggered from a flow step (`- action: utter_current_balance`) ran through the action machinery, and the built-in patterns that repaired our conversations ran actions of their own. What Day 8 adds is not the concept — it is *your code* joining a system of actions that already exists.

A custom action is a Python class with two obligations:

```python
class ActionSomething(Action):
    def name(self) -> Text:              # how YAML refers to it
        return "action_something"

    def run(self, dispatcher, tracker, domain) -> List[Dict]:
        ...                              # read state, do work
        return []                        # events that change the conversation
```

The `run` signature is the whole API, and today's three actions each lean on a different part of it:

Parameter	What it is	Today's showcase
`tracker`	read-only view of the conversation: slots, sender id, latest message	`action_confirm_appointment`
`dispatcher`	the output channel: text, responses, buttons, custom JSON	`action_send_statement_link`
return value	**events** — facts re-entering the conversation, e.g. `SlotSet`	`action_fetch_balance`

(One more trick for later days: returning a class whose `name()` matches a *default* action — for example `action_session_start` — replaces that default with your version. Override-by-name is how session-start logic gets customized.)

2. The mock bank

A bank assistant needs a bank. `lumera-fastapi-server/` is a ~70-line FastAPI app that plays core banking for the rest of the course:

- `GET /health` — public liveness check
- `GET /v1/balance?account_type=...` — per-account balances, bearer-token protected
- `POST /v1/appointments` — records a booking, returns a reference like `LUM-0001`, token protected

It is deliberately tiny — small enough to read in class, and small enough to **kill on purpose** when we test failure. From this lesson's folder:

```
make setup               # one-time: builds both venvs (server + assistant, with rasa-pro + requests)
make run-fastapi-server  # terminal 1 — the bank, on :8000
```

Check it responds — note the token, and that a wrong one is rejected:

```
curl http://localhost:8000/health
# {"status":"ok"}
curl -H "Authorization: Bearer test_token" "http://localhost:8000/v1/balance?
account_type=savings"
# {"account_type":"savings","balance":2087.5}
curl -H "Authorization: Bearer wrong" "http://localhost:8000/v1/balance?
account_type=savings"
# {"detail":"Invalid token"}
```

3. Action 1 — `action_fetch_balance` : events out

The balance stub dies first. Create `actions/fetch_balance.py` :

```python
import os
from typing import Any, Dict, List, Text

import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher


class ActionFetchBalance(Action):
    def name(self) -> Text:
        return "action_fetch_balance"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        api_base = os.environ.get("CORE_BANKING_URL")
        token = os.environ.get("CORE_BANKING_TOKEN")
        account_type = tracker.get_slot("account_type")

        try:
            response = requests.get(
                f"{api_base}/v1/balance",
                params={"account_type": account_type},
                headers={"Authorization": f"Bearer {token}"},
                timeout=5,
            )
            response.raise_for_status()
            balance = response.json()["balance"]
            return [
                SlotSet("current_balance", balance),
                SlotSet("balance_fetch_ok", True),
            ]
        except (requests.RequestException, KeyError, TypeError, ValueError):
            return [
                SlotSet("current_balance", None),
                SlotSet("balance_fetch_ok", False),
            ]
```

Read it line by line — every line is a policy decision:

- **The URL and token come from the environment.** No credential appears in source, YAML, model artifacts, or prompts. The LLM never sees them.
- **`timeout=5` is not decoration.** A hung backend becomes a *designed* failure in five seconds instead of a silent wait.
- **The action does not decide what to say.** It returns facts as events: the balance when available, and whether the fetch worked (`balance_fetch_ok`, a status flag the flow will branch on).

- **The failure path clears `current_balance`** — a stale balance from an earlier successful call must not survive a later failure.
- Remember the `controlled` mapping from Day 7: a `SlotSet` event from a custom action is exactly the writer that mapping permits. The LLM still cannot invent a balance; now *the bank* supplies it.

Three YAML touches wire it in. The domain (`domain/check_balance.yml`) gains the status slot, the failure response, and — new requirement — the action's **registration**:

```yaml
slots:
  # ... account_type and current_balance as on Day 7 ...
  balance_fetch_ok:
    type: bool
    mappings:
      - type: controlled

responses:
  # ... existing responses ...
  utter_balance_service_down:
    - text: "We're sorry, the balance service is unavailable right now. Please try
again later."

actions:
  - action_fetch_balance
```

One string, three places: the value of `name()`, the entry under `actions:`, and the step in the flow. They must match exactly.

And the flow (`data/flows.yml`) replaces the lie with the action plus a branch on its status flag:

```yaml
  check_balance:
    name: check account balance
    description: Look up the current balance of one of the customer's Banca Lumera
accounts.
    steps:
      - collect: account_type
        description: "the account to check: checking or savings"
      - action: action_fetch_balance
        next:
          - if: slots.balance_fetch_ok
            then: show_balance
          - else:
              - action: utter_balance_service_down
                next: END
      - action: utter_current_balance
        id: show_balance
```

The action *could* utter the apology itself — but that would bury an unhappy path inside Python. Failure is a conversation decision, so it lives in the flow, reviewable and visible in the Inspector diagram. That is "logic in flows, work in actions" in one screen of YAML.

Run it — and notice what we didn't need

Where does this Python *execute*? Look at `endpoints.yml`, unchanged since Day 7:

```
action_endpoint:
  actions_module: "actions"
```

This is the **in-process** wiring: Rasa imports the `actions` package and runs your code inside the assistant's own process. No extra server, no extra terminal — for development, this is all it takes. (The other wiring gets its moment in §6.)

```
rasa train
rasa inspect     # keep the mock bank running in terminal 1
```

Ask **"what's my savings balance?"**:

> **bot:** Your savings account balance is €2087.5.

The number now comes from the API — ask for checking and you get a *different* number (€1250.0), which the stub could never do. In the Inspector's slot pane, watch `current_balance` and `balance_fetch_ok` appear together right after the action step runs.

4. Action 2 — `action_confirm_appointment`: the tracker

Second lie: the booking that never landed. This action's job is mostly *reading* — everything the conversation collected, pulled back out of the tracker and shipped to the bank. Create `actions/confirm_appointment.py`:

```python
import os
from typing import Any, Dict, List, Text

import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher


class ActionConfirmAppointment(Action):
    def name(self) -> Text:
        return "action_confirm_appointment"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        api_base = os.environ.get("CORE_BANKING_URL")
        token = os.environ.get("CORE_BANKING_TOKEN")

        # Everything the conversation collected, read back from the tracker.
        payload = {
            "customer_id": tracker.sender_id,
            "branch_city": tracker.get_slot("branch_city"),
            "topic": tracker.get_slot("appointment_topic"),
            "preferred_time": tracker.get_slot("preferred_time"),
        }

        try:
            response = requests.post(
                f"{api_base}/v1/appointments",
                json=payload,
                headers={"Authorization": f"Bearer {token}"},
                timeout=5,
            )
            response.raise_for_status()
            reference = response.json()["reference"]
            return [
                SlotSet("booking_reference", reference),
                SlotSet("booking_ok", True),
            ]
        except (requests.RequestException, KeyError, TypeError, ValueError):
            return [
                SlotSet("booking_reference", None),
                SlotSet("booking_ok", False),
            ]
```

The tracker tour, in three reads:

- `tracker.get_slot("...")` — the slots the flow collected on Day 7, one call each. The flow gathered them; the action consumes them.
- `tracker.sender_id` — the conversation's stable identifier, here doubling as a (mock) customer id. In production this is where your channel's authenticated user id surfaces.
- `tracker.latest_message` — not used in the payload, but print it once in class: it holds the raw text of the user's last message (the "yes please, book it") plus the structured metadata the pipeline attached to it. When an action needs to know *how* something was said, this is where to look.

Wiring, same three touches — shown in full, as with `check_balance` above. `domain/book_appointment.yml` gains the two controlled slots, the failure response `utter_booking_failed`, and the action registration; and the existing `utter_appointment_booked` now interpolates a real artifact of the integration:

```yaml
slots:
  # ... branch_city, appointment_topic, preferred_time, appointment_confirmed as on Day
7 ...
  booking_ok:
    type: bool
    mappings:
      - type: controlled
  booking_reference:
    type: text
    mappings:
      - type: controlled

responses:
  # ... existing appointment responses ...
  utter_appointment_booked:      # Day 7's response, now carrying the reference
    - text: "Done — your appointment is booked, reference {booking_reference}. You'll
receive a confirmation shortly."
  utter_booking_failed:
    - text: "I couldn't reach the booking system, so nothing was booked. Please try
again in a few minutes."

actions:
  - action_confirm_appointment
```

In the flow, the confirmation branch stops being conversational theater — on "yes" it runs the action and branches on the outcome:

```
    - collect: appointment_confirmed
      description: whether the customer confirms the recap of the appointment
      next:
        - if: slots.appointment_confirmed
          then:
            - action: action_confirm_appointment
              next:
                - if: slots.booking_ok
                  then:
                    - action: utter_appointment_booked
                      next: END
                - else:
                    - action: utter_booking_failed
                      next: END
        - else:
            # ... the Day 7 decline branch, unchanged ...
```

Retrain and run the full booking. The recap and confirmation are exactly as yesterday — but the closing line now carries the bank's reference:

> **bot:** Done — your appointment is booked, reference LUM-0002. You'll receive a confirmation shortly.

That reference was minted by the mock API, which now has the appointment on record. The conversation produced a durable effect in another system — that is the integration boundary, crossed.

5. Action 3 — `action_send_statement_link`: the dispatcher

Third face: an action whose whole job is *output*. New small capability — the customer wants their account statement, and the answer is a **deeplink built from slot values**. Create `actions/send_statement_link.py`:

```python
from typing import Any, Dict, List, Text

from rasa_sdk import Action, Tracker
from rasa_sdk.executor import CollectingDispatcher


class ActionSendStatementLink(Action):
    def name(self) -> Text:
        return "action_send_statement_link"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        account_type = tracker.get_slot("account_type")
        url = f"https://banca-lumera.example/statements/{account_type}"

        dispatcher.utter_message(
            text=f"Here is the statement of your {account_type} account:"
        )
        dispatcher.utter_message(
            json_message={
                "type": "deeplink",
                "url": url,
                "label": f"Download {account_type} statement (PDF)",
            }
        )
        return []
```

`dispatcher.utter_message(...)` is the output multitool; its keyword decides the shape. `text=` sends plain words; `response="utter_x"` triggers a domain response by name (keeping wording in YAML even when Python decides the moment); `buttons=` attaches choices; and `json_message=` sends an **arbitrary custom payload** — the channel passes it through untouched for the client application to render. That last one is how structured things — deeplinks, cards, widgets — travel from an action to a frontend. Note this action returns `[]`: it changed nothing in the conversation state; it only *said* things.

Wire it with a fourth flow (in `data/flows.yml`) and a two-line domain file (`domain/download_statement.yml`) that just registers the action:

```
  download_statement:
    name: download account statement
    description: Send the customer a link to download the statement of one of their
accounts.
    steps:
      - collect: account_type
        description: "the account whose statement the customer wants: checking or
savings"
      - action: action_send_statement_link
```

And `domain/download_statement.yml` in full — no new slot or response, it only registers the action:

```
version: "3.1"

actions:
  - action_send_statement_link
```

No new slot: `download_statement` collects the same `account_type` that `check_balance` uses — slots belong to the conversation, not to a flow. (If you checked a balance earlier in the session, the statement flow won't even ask.)

Retrain and try **"I need the statement of my savings account"** — over the REST API the answer comes back as one text message plus one `custom` payload:

```
{"text": "Here is the statement of your savings account:"}
{"custom": {"type": "deeplink",
            "url": "https://banca-lumera.example/statements/savings",
            "label": "Download savings statement (PDF)"}}
```

A parameterized deeplink, assembled from conversation state, delivered as machine-readable JSON: precisely the handoff a web or app frontend consumes.

6. The second wiring: an external action server

The in-process module served us three actions with zero ceremony. Now see the other shape — because in production, action code usually does **not** live inside the assistant process. Two terminals:

```
make run-actions-server    # terminal 2 — rasa run actions, an HTTP server on :5055
```

and flip `endpoints.yml` from module to URL:

```
action_endpoint:
  url: "http://localhost:5055/webhook"
```

Restart the assistant (`make run-rasa`) and run any balance check — identical behavior, different anatomy: each action step is now an HTTP round-trip to a separate process.

Why would you want this?

- **Isolation** — action code crashes, upgrades and scales without touching the assistant; heavy dependencies stay out of the Rasa process.
- **Credential separation** — try it: the assistant terminal needs *no* `CORE_BANKING_TOKEN` anymore. Only the action server holds bank credentials; the model-facing process literally cannot leak what it does not have.
- **Team boundary** — the action server is a plain Python service your backend team can own, test, and deploy like any other.

Both wirings are first-class; the choice is per-project pragmatics. **For this course we now switch back to the in-process module** — one terminal fewer every session, and nothing in the coming lessons depends on the boundary. Restore:

```
action_endpoint:
  actions_module: "actions"
```

When the course reaches production topics, the external server returns as the proper shape for the integration boundary.

7. Failure theater

Two endings, staged deliberately. Keep the assistant running.

The designed failure. Stop the mock bank (Ctrl-C in terminal 1) and ask for a balance:

> **bot:** We're sorry, the balance service is unavailable right now. Please try again later.

The action's `except` caught the connection error, `balance_fetch_ok` came back `false`, and *our* flow branch produced a branded, specific, recoverable answer. Try the booking too — it has its own designed failure:

> **bot:** I couldn't reach the booking system, so nothing was booked. Please try again in a few minutes.

("Nothing was booked" is not filler — the action guarantees it: no reference, no success flag, no false promise.)

The unhandled failure. Now sabotage the action — add a `raise RuntimeError("boom")` as the first line of `action_fetch_balance.run()`, restart, and ask again:

> **bot:** Sorry, I am having trouble with that. Please try again in a few minutes. **bot:** Okay, stopping pattern_collect_information.

The exception crossed the SDK boundary, Rasa cancelled the active flow, and a built-in repair pattern (`pattern_internal_error`) produced a generic apology. Compare the two transcripts: same broken backend, but one answer is yours and one is the framework's last resort. The `try`/`except` and the status-flag slot are the difference. **Remove the sabotage line before moving on.**

8. Where we landed

```
day-08-custom-actions-and-integration/
├── Makefile                      # process runner (bank / action server / rasa)
├── lumera-fastapi-server/        # the mock core-banking API
└── lumera-assistant/
    ├── actions/
    │   ├── fetch_balance.py          # events out  (SlotSet + status flag)
    │   ├── confirm_appointment.py    # tracker in  (slots, sender_id → POST)
    │   └── send_statement_link.py    # dispatcher out (text + json_message deeplink)
    ├── data/flows.yml            # check_balance branches; booking lands; +
download_statement
    ├── domain/                   # + controlled result/status slots, + actions:
registrations
    └── endpoints.yml             # in-process module (external URL demoed, reverted)
```

The division of labor, now in running code:

- The **domain** declares what may exist: user-fillable inputs, controlled backend facts, responses, registered actions.
- The **flow** owns the conversation logic: collect, run the action, branch on the status flag.
- The **action** does the work: read env credentials, call the backend with a timeout, return events — or speak through the dispatcher.
- **`endpoints.yml`** picks the process boundary: in-process for development comfort, external URL for isolation and credential separation.

Day 7's two lies are gone: the balance is fetched, the booking lands and returns a reference. What the assistant *says* is still entirely YAML — Python only ever supplied facts and payloads. That discipline is what will keep this codebase reviewable as it grows.